DATA 260
HOMEWORK 9

Redis Semantic Caching and Self-Improving Multi-Agent System

Shoury Ambarish Parab
SID4: 7085
Repository: https://github.com/shoddy16/data260-7085

Project configuration

Setting	Verified value
PREFIX	s7085
SEED / VERIFY_SEED	7085 / 267085
DOMAIN_ID	5
PORT_BASE / HW9_PORT	8085 / 8086
Chat / embedding	llama3.2:3b / nomic-embed-text
Embedding dimension	768
Backend	FastAPI, SQLAlchemy, Redis Stack, Ollama

Contents

1. Project Context and Architecture

2. Part 1 — Redis-Backed Restaurant View Counter

3. Part 2 — Redis Semantic Response Cache

4. Part 3 — Frozen Dataset and Threshold Economics

5. Stale-Answer Experiment and Version Invalidation

6. Part 4 — Self-Improving Multi-Agent System

7. Testing and Regression Verification

8. Runtime Evidence and Required Artifacts

9. Reproducibility

10. AI Use Disclosure

11. Final Requirement Checklist

12. Conclusion

1. Project Context and Architecture

I built HW9 by extending the restaurant-inspection application from the earlier homework instead of starting a separate project. I added four connected features: a Redis view counter, a semantic cache for the existing question-answering endpoint, a threshold experiment to measure cache behavior, and a two-agent workflow that can learn and reuse lessons.

The application uses FastAPI to handle requests, SQLAlchemy to work with the relational database, Redis Stack for counting and vector search, and local Ollama models for chat and embeddings. The project is configured for the restaurant-inspection domain, and Redis keys use the required s7085 prefix.

There are two main runtime paths. When a restaurant is opened, its Redis view count is incremented and synchronized with the database every ten views. For a question, the application creates an embedding and uses Redis HNSW KNN to find a similar cached query. The cache then checks the threshold and the query safeguards before returning a saved answer or calling the existing agent pipeline.

Configuration used for the reported runs: SID4 7085; PREFIX s7085; SEED 7085; VERIFY_SEED 267085; DOMAIN_ID 5; PORT_BASE 8085; HW9 API port 8086; chat model llama3.2:3b; embedding model nomic-embed-text; embedding dimension 768; Redis host port 6380.

Setting	Value
Domain	Local restaurant inspections
Redis prefix	s7085
API port	8086
Redis host port	6380
Chat model	llama3.2:3b
Embedding model	nomic-embed-text (768D)
Services	FastAPI, SQLAlchemy, Redis Stack, Ollama

Table 1. Project configuration used for HW9.

1.1 Request Flow and Component Boundaries

When an authenticated user opens a restaurant, the counter service creates a key for that restaurant and calls Redis INCR. Redis increments the value atomically. The service updates the relational database only when the count is divisible by ten. The route still returns the current count on the other requests.

The Q&A flow starts at POST /hw9/ask. The route sends the question to the semantic cache and provides a callback to the existing answer agent. The cache creates an embedding, runs a Redis KNN query, and checks the closest result. If the result meets the threshold and passes the additional checks, the saved answer is returned without running the agent. Otherwise, the callback generates an answer and the cache stores it.

I kept the self-improvement experiment separate from the live cache path. It uses reference facts to check answers and a lesson file to test how evaluator feedback and saved lessons affect correctness. This also keeps the local agent timings separate from timings that include Ollama network calls.

Component	Responsibility
ViewCounter	Atomic Redis increments and tenth-view database sync
SemanticCache	Embedding, Redis KNN, similarity, metadata and invalidation
Q&A route	Connects cache hit/miss behavior to the existing agent callback
Answering Agent	Produces an initial answer and applies relevant lessons
Evaluator Agent	Checks answers against reference facts and supplies feedback
Experiment runner	Compares four variants on the same held-out questions

Table 2. Responsibilities of the main HW9 components.

2. Part 1 — Redis-Backed Restaurant View Counter

For Part 1, the counter needs to increase in Redis whenever a restaurant is accessed, use the assigned key prefix, and write the count to the relational database every tenth view. I implemented this in the HW9 service layer and connected it to the restaurant view route.

Each counter key follows this format: s7085:views:restaurant:{restaurant_id}. Including the restaurant ID keeps the counts separate for each restaurant. Redis INCR handles each increment atomically. This is useful when requests arrive at the same time because the application does not have to read a value, add one, and write it back in separate steps.

The service checks count % 10 == 0. On the tenth, twentieth, thirtieth, and every later tenth view, it loads the matching restaurant row and saves the current count. Redis handles the increments between those database updates, so the database is not written on every request.

Source excerpt — code_counter.png

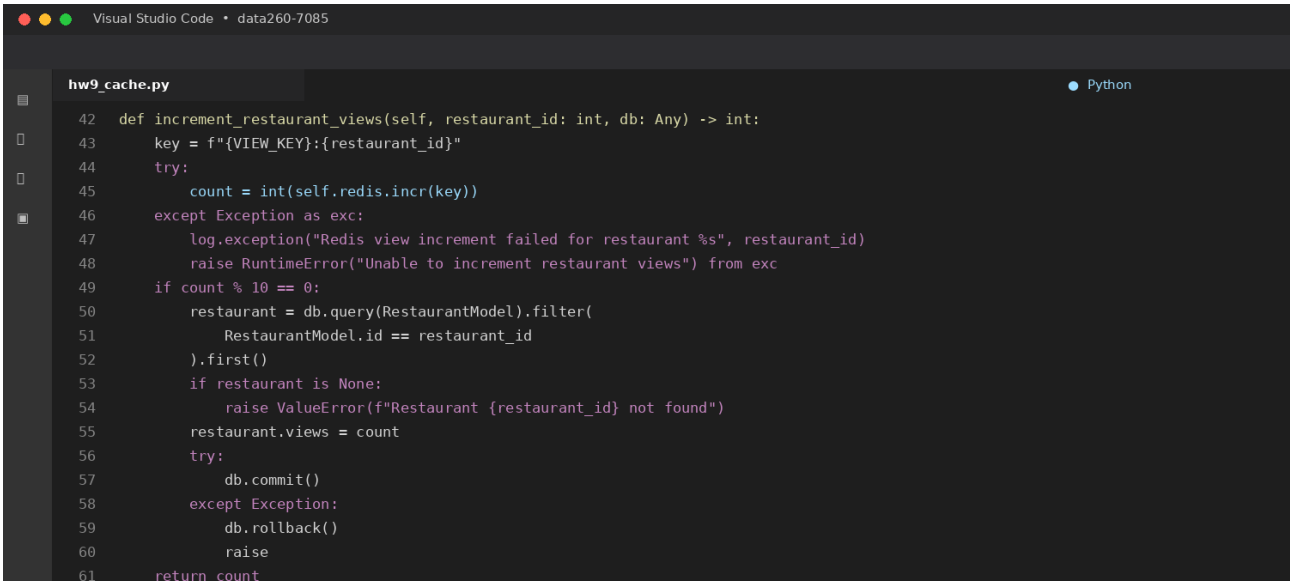


Figure 1. View-counter source excerpt from hw9_cache.py.

2.1 Error Handling and Database Synchronization

The Redis increment has error handling around it. If Redis raises an exception, the service logs the error and raises a RuntimeError. The API can then return a service-unavailable response instead of returning a count that was not successfully incremented.

When the count reaches a tenth-view boundary, the service checks that the restaurant row exists. If it does not, the service raises a ValueError and the route returns HTTP 404. If the database commit fails, the SQLAlchemy session is rolled back before the error is raised again. This keeps the session from being left in a failed transaction state.

The route uses dependency injection for the database session and current user. Its response includes the restaurant ID and current view count. Redis and runtime errors return HTTP 503, while a restaurant that does not exist returns HTTP 404.

Source excerpt — code_counter_route.png

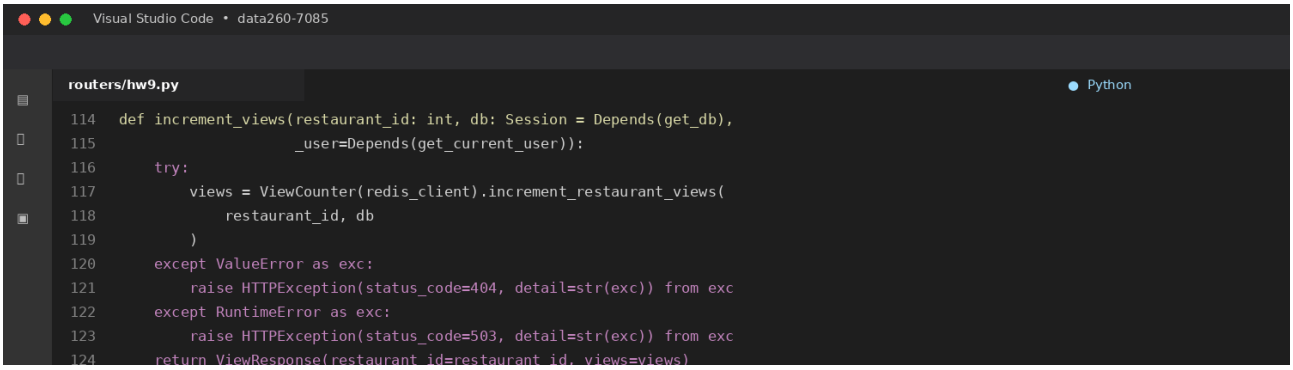


Figure 2. API route excerpt for restaurant view counting.

2.2 Live Counter Results and Concurrency

In the live Redis/MySQL run, restaurant 1 returned counts 1, 2, 10, 20, and 21. Its Redis key was s7085:views:restaurant:1. After view 21, the database still showed 20. That is the expected result because the database is updated at every tenth view, not after every request.

I also ran a concurrent test with restaurant 2. It produced 20 unique Redis counts, and both the final Redis count and database value were 20. This run shows that the atomic Redis increment handled the simultaneous requests and that the tenth-view database updates worked as expected.

The focused HW9 tests also check that different restaurant IDs have separate counts and that the service handles a Redis client failure. The live run covers the Redis/MySQL flow, while the unit tests make the error and boundary checks repeatable.

Observation	Result
Redis key	s7085:views:restaurant:1
Observed counts	1, 2, 10, 20, 21
Database after view 21	20
Tenth-view sync	Confirmed
Concurrent unique counts	20
Concurrent Redis count	20
Concurrent database count	20

Table 3. Live counter and concurrency evidence.

3. Part 2 — Redis Semantic Response Cache

The semantic cache runs at the start of the existing /hw9/ask endpoint. For each entry, it stores the original question, its embedding, the final answer, a query signature, the restaurant scope, a data-version marker, and a creation timestamp. These fields let the cache search for similar questions and check whether a saved answer still applies.

The embedding model is nomic-embed-text, which returns 768-dimensional vectors. Redis Stack stores the cache records and indexes the vectors with HNSW using cosine distance. The index is named s7085:semantic_cache_idx_v768, and the vector field uses the explicit alias embedding_vector.

The final index uses the explicit embedding_vector alias and a 768-dimensional FLOAT32 vector configuration, while keeping the existing JSON records. This configuration is used by the qualifying experiment, where every threshold row records redis_knn=true.

Source excerpt — code_knn.png

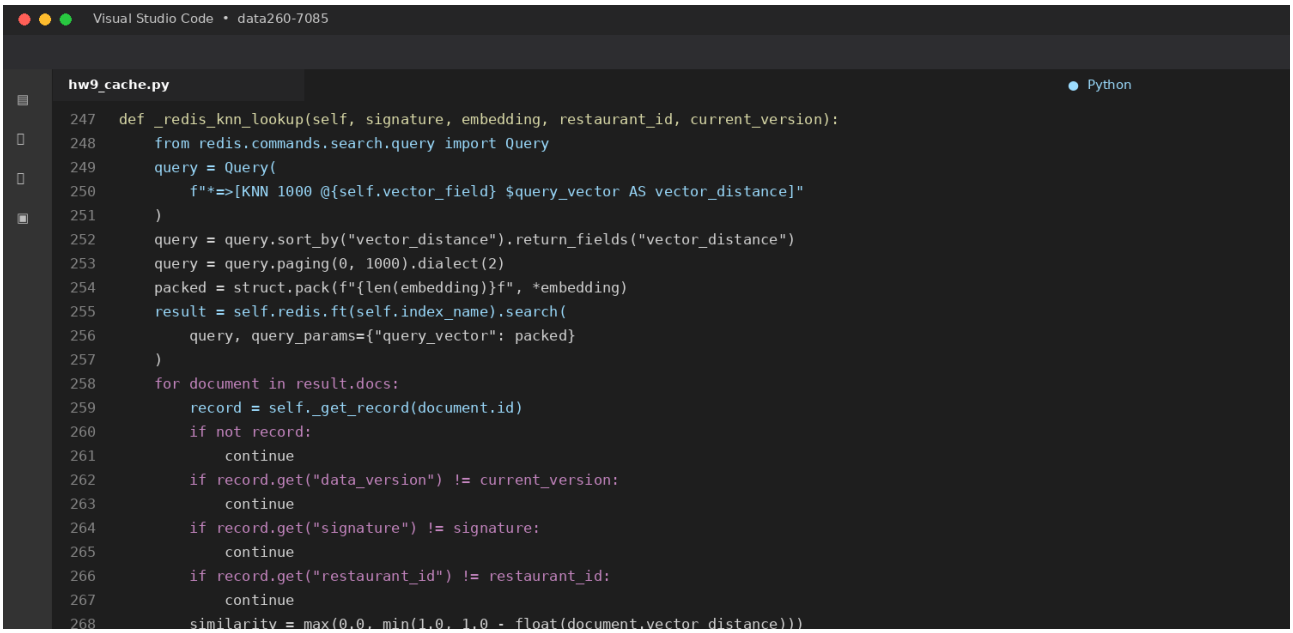


Figure 3. Redis HNSW KNN lookup source excerpt.

3.1 Similarity, Threshold, and Cache Decisions

Redis Search returns vector_distance for the cosine index. The implementation converts it to similarity with 1 - distance and keeps the result between zero and one. The configured threshold decides whether the closest candidate is similar enough to be considered for reuse.

The threshold is only one part of the decision. The cache also checks the query signature, restaurant ID, and current data version. It rejects a candidate if the signature is different, the candidate belongs to another restaurant, or the stored answer uses an older data version.

This is important because similar wording does not always mean that two database questions are asking for the same thing. Two questions can use almost all the same words but have different filters or comparison conditions. The extra checks help prevent an answer for one logical query from being reused for another.

Condition	Purpose
Similarity >= threshold	Rejects weak semantic matches
Query signature	Protects important query semantics such as comparison direction
Restaurant ID	Prevents cross-restaurant answer reuse
Data version	Rejects entries written before a record update
Redis KNN backend	Provides nearest-neighbor candidate retrieval

Table 4. Cache acceptance safeguards.

3.2 Integration with the Existing Agent Endpoint

The /hw9/ask route gives the cache a callback to run the existing answer system. This keeps the cache separate from the details of the agent itself. The cache decides whether the callback needs to run, while the callback continues to use the answer path carried over from HW8.

When there is a cache hit, the endpoint returns the saved answer and skips the answer agent. When there is a miss, the callback runs and the final answer is saved with its embedding and metadata. The response also identifies the result as uncached. This allows repeated or sufficiently similar questions to avoid another answer-generation call.

The live checks included an exact duplicate with similarity 1.0 and a paraphrase with similarity about 0.9386; both were served from cache. A question with a different logical condition and an unrelated question were misses. The stale-version, empty-index, and concurrent lookup checks also used the Redis KNN backend.

Source excerpt — code_route.png

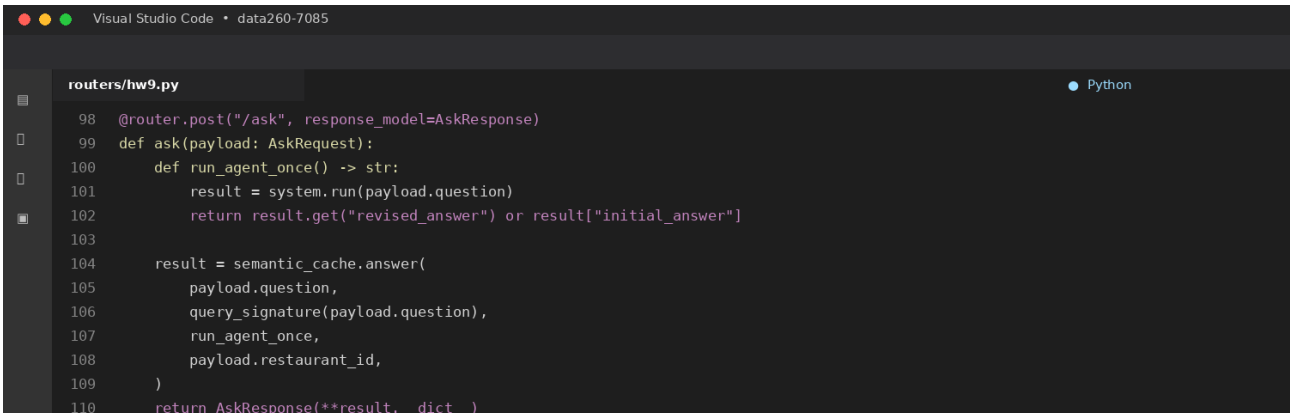


Figure 4. Q&A route and cache callback integration.

3.3 Live Semantic Cache Validation

The exact-match test returned the original cached query with similarity 1.0. The paraphrase 'List restaurants with scores under 70' matched the seed query 'Which restaurants scored below 70?' with similarity approximately 0.9386. The cache returned the stored answer without executing the agent callback.

The logical-difference and unrelated cases were rejected. The logical-difference case asked about scores above 70 rather than below 70. The unrelated case asked for an explanation of binary search. Both returned misses, so the answer path was allowed to execute.

The validation also incremented the restaurant data-version marker and confirmed that an older cached record was no longer accepted. An isolated empty index returned a miss. Eight concurrent queries all reported the Redis KNN backend and all eight were served as cache hits in that controlled concurrency check.

Case	Observed behavior
Exact duplicate	Hit; similarity 1.000000
Paraphrase	Hit; similarity 0.938599
Logical difference	Miss
Unrelated question	Miss
Stale version	Rejected after version increment
Empty index	Miss
Concurrent lookup	8 of 8 used Redis KNN

Table 5. Live semantic-cache validation cases.

4. Part 3 — Frozen Dataset and Experimental Method

For the threshold experiment, I used the frozen cache_cases.yaml dataset, which was committed before the qualifying sweep. It contains 40 labeled restaurant-inspection questions split into four equal groups. This setup measures both useful cache reuse and cases where a similar question should not reuse an answer.

The dataset has ten exact duplicates expected to hit, ten paraphrases expected to hit, ten questions with similar wording but different answers expected to miss, and ten unrelated questions expected to miss. The expected labels were set before the qualifying experiment and stayed the same throughout the sweep.

The same 40 cases were evaluated at thresholds 0.75, 0.80, 0.85, 0.90, and 0.95. The embedding model was the live Ollama nomic-embed-text model, with 768 dimensions, and candidate retrieval used Redis HNSW KNN. The raw qualifying result file is threshold_sweep_qualifying_knn.json.

Dataset group	Cases	Expected outcome
Exact duplicates	10	Hit
Paraphrases	10	Hit
Similar wording, different answer	10	Miss
Unrelated questions	10	Miss
Total	40	20 expected hits / 20 expected misses

Table 6. Frozen dataset composition.

4.1 Metrics and Cost Model

I calculated hit rate by dividing the number of cache hits by all 40 cases. False-hit rate uses the 20 cases labeled as expected misses. This denominator shows the share of questions where the cache should have rejected reuse but returned a saved answer instead.

The cost model assigns one unit to a cache miss and eight units to a false hit. A miss makes the system do more work, while a false hit can return an incorrect answer and confuse the user or create extra support work. For that reason, I gave a false hit a higher cost.

Total cost is (40 - hits) × 1 + false hits × 8. I used the same formula for all five thresholds. These are experiment cost units, not actual dollar amounts.

Metric	Formula
Hit rate	hits / 40
False-hit rate	false hits / 20
Miss cost	1 unit per miss
False-hit cost	8 units per false hit
Total cost	(40 - hits) × 1 + false_hits × 8

Table 7. Evaluation metrics and cost model.

4.2 Qualifying Five-Threshold Results

The results show the trade-off between more cache hits and fewer incorrect cache hits. A lower threshold accepts more candidates, which increases the number of hits but also allows more false hits. A higher threshold is more selective, so it reduces false reuse while increasing misses.

At 0.75, the cache returned 29 hits, including nine false hits. At 0.95, it returned 13 hits, including three false hits. The false-hit rate dropped from 45% to 15% of the expected-miss cases, and the total cost dropped from 83 to 51 under the selected cost model.

The 0.85 starting threshold returned 27 hits and eight false hits. Its cost was 77, slightly higher than the cost of 76 at 0.80. This shows why I measured the thresholds instead of assuming that the initial 0.85 setting was the best choice.

Threshold	Hits	Misses	Correct hits	False hits	Missed expected hits	Hit rate	False-hit / 20	Mean ms	Cost
0.75	29	11	20	9	0	0.725	0.450	2537.92	83
0.80	28	12	20	8	0	0.700	0.400	103.61	76
0.85	27	13	19	8	1	0.675	0.400	151.57	77
0.90	22	18	16	6	4	0.550	0.300	184.15	66
0.95	13	27	10	3	10	0.325	0.150	240.44	51

Table 8. Qualifying live Redis KNN threshold sweep.

4.3 Interpreting the Threshold Trade-Off

The hit-rate trend declines as the threshold rises: 0.725 at 0.75, 0.700 at 0.80, 0.675 at 0.85, 0.550 at 0.90, and 0.325 at 0.95. The false-hit rate also declines: 0.450, 0.400, 0.400, 0.300, and 0.150 respectively.

A higher threshold is more selective, but it also reduces cache reuse. At 0.95, ten cases labeled as expected hits were missed. Those questions go through the normal agent path instead of being answered from cache. Under this cost model, 0.95 gives the lowest cost and the fewest false hits, with fewer useful cache hits as the trade-off.

The result is based on this 40-question dataset and the cost assumptions used in the experiment. A different question mix or a different penalty for incorrect answers could change which threshold is the best choice. For the cases tested here, the measurements support choosing 0.95.

Threshold	Hit rate	False-hit rate	Cost
0.75	72.5%	45.0%	83
0.80	70.0%	40.0%	76
0.85	67.5%	40.0%	77
0.90	55.0%	30.0%	66
0.95	32.5%	15.0%	51

Table 9. Hit and false-hit rates by threshold.

4.4 False-Hit Analysis

The qualifying sweep included a false hit caused by two questions with very similar wording but different numeric conditions. The cached question was 'Which restaurant scored above 90?' and the incoming question was 'Which restaurants scored above 70?' Their similarity was 0.869723975658 at threshold 0.75.

The cached answer was correct for the above-90 question, but it was not correct for the incoming above-70 question. The dataset label therefore expected a miss. The embedding treated the questions as close because both mention restaurants, scores, and an above-threshold comparison. However, changing 90 to 70 changes which restaurants belong in the answer.

This example shows that vector similarity by itself is not enough to decide whether two questions can share an answer. A cache should also compare structured filters such as numeric thresholds, comparison operators, dates, and entity scope before returning a saved result.

Field	Recorded value
Cached question	Which restaurant scored above 90?
Incoming question	Which restaurants scored above 70?
Similarity	0.869723975658
Threshold	0.75
Cached answer	Verified answer for the above-90 question
Correct action	Miss; answer the above-70 question normally

Table 10. Full qualifying false-hit example.

4.5 Latency Interpretation

The mean latency at threshold 0.75 was 2,537.92 ms, which was much higher than the other threshold means. I kept this value as recorded in the qualifying results rather than changing or excluding it.

The latency result is useful to read alongside the rest of the sweep because it includes the full lookup path, not just the Redis query. The table preserves the measured value so the threshold comparison reflects the recorded experiment.

The reported latency is end-to-end lookup time for this experiment. It includes embedding generation and Redis KNN lookup, so it is not a measurement of Redis KNN alone. The other threshold means were 103.61 ms, 151.57 ms, 184.15 ms, and 240.44 ms.

Threshold	Mean latency
0.75	2537.92 ms
0.80	103.61 ms
0.85	151.57 ms
0.90	184.15 ms
0.95	240.44 ms

Table 11. End-to-end latency values from the qualifying sweep.

4.6 Threshold Selection

Based on the required cost model, I selected 0.95 because it had the lowest measured total cost: 51. It also produced the fewest false hits in the sweep, with three compared with nine at 0.75.

There is a clear trade-off at 0.95. The cache returned 13 hits out of 40 and missed ten cases that were labeled as expected hits. It also recorded three false hits, which is why the other correctness checks remain important alongside the threshold.

I kept the query-signature, restaurant-scope, and data-version safeguards enabled. For questions involving numeric score comparisons, date ranges, or other structured conditions, I would also require the predicates to match before reusing an answer. The measured threshold works together with these query checks.

Decision factor	0.95 result
Total cost	51 (lowest)
False hits	3
False-hit rate	15% of expected-miss cases
Cache hits	13 of 40
Missed expected hits	10
Decision	Select 0.95 under this cost model, with safeguards

Table 12. Rationale for the selected threshold.

5. Stale-Answer Experiment and Version Invalidation

A cached answer can be correct when it is first stored and become outdated after the restaurant record changes. I tested this using a MySQL/Redis sequence to show how a stale answer is served and how a data-version update prevents it from being reused.

The first query returned location=San Jose and saved that answer in the cache. The next identical query returned the cached answer. I then changed the restaurant location in the database. Before invalidation, another query still returned the old cached location. This demonstrates the stale hit required by the assignment.

After I incremented the restaurant's data-version marker, the same query no longer accepted the old cache entry. The answer path ran again and returned location=HW9-Updated-City. There was no need to clear the entire cache; the version change applied to the affected restaurant.

Source excerpt — code_version.png

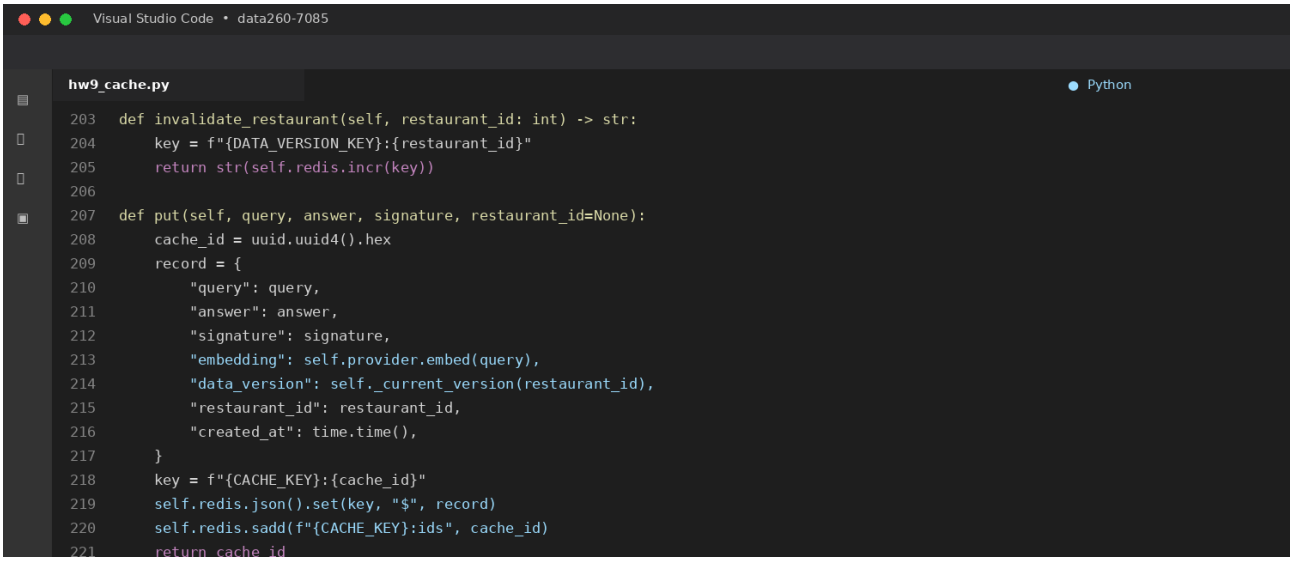


Figure 5. Data-version marker and cache-record storage excerpt.

5.1 Stale-Answer Evidence and Why Versioning Helps

The live evidence shows the first uncached answer as location=San Jose. The stale query before invalidation returned the same answer with is_cached=true and similarity 1.0. After the database update but before the version increment, the cache continued to serve the old answer. After invalidation, the response changed to location=HW9-Updated-City and is_cached=false.

Version markers are more targeted than clearing the whole cache because they invalidate entries for one restaurant while leaving answers for other restaurants available. The important part is to increment the correct restaurant's version whenever its relevant data changes. This keeps old answers from being treated as current.

Stage	Answer	Cached?	Agent executed?
Initial query	location=San Jose	No	Yes
Repeated query	location=San Jose	Yes	No
After DB update, before invalidation	location=San Jose (stale)	Yes	No
After version increment	location=HW9-Updated-City	No	Yes

Table 13. Stale-answer lifecycle from live MySQL/Redis evidence.

6. Part 4 — Self-Improving Multi-Agent System

The self-improvement component has two agents: an Answering Agent and an Evaluator Agent. The Answering Agent creates an answer using the reference facts and any relevant saved lessons. The Evaluator Agent checks the answer against the reference dataset and either accepts it or gives feedback for one revision.

The workflow allows a maximum of two answer attempts. The evaluator checks the first answer. If it passes, the system returns it. If it needs revision, the Answering Agent receives the feedback and produces one updated answer. The evaluator checks that second answer, and then the workflow stops. This keeps the repair loop bounded.

The reference dataset contains the correct restaurant facts. The training, validation, and test datasets contain input questions only. Keeping the expected answers out of those input files prevents answer leakage and keeps the held-out test set separate from lesson creation and validation.

Source excerpt — code_agents.png

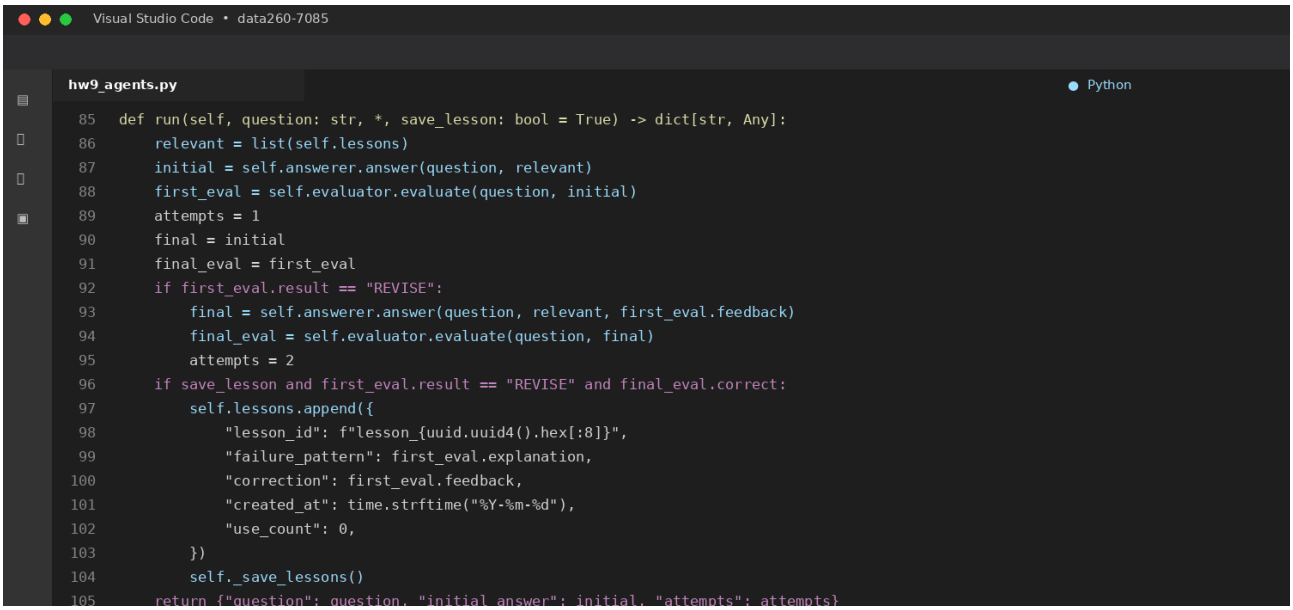


Figure 6. Two-attempt repair and lesson creation excerpt.

6.1 Dataset Separation and Ground Truth

The agent experiment uses reference_data.json as the source of correct restaurant facts. The training_data.json, validation_data.json, and test_data.json files contain questions only. The Evaluator Agent checks each generated answer against the reference facts.

Training questions are used while developing the agent behavior. Validation questions check whether a proposed lesson fixes known failure patterns without changing answers that were already correct or causing unrelated questions to receive unsupported answers. The test questions are kept for the final four-variant comparison.

I did not use the test set to create lessons or tune the answer prompt. All four variants use the same ten held-out questions, so the comparison is based on the same inputs.

Dataset	Purpose	Contains expected answers?
reference_data.json	Ground-truth domain facts	Yes — reference facts
training_data.json	Development questions	No
validation_data.json	Lesson acceptance checks	No
test_data.json	Held-out final evaluation questions	No
lessons.json	Reusable failure patterns and corrections	Not applicable

Table 14. Data separation in the agent experiment.

6.2 Controlled Repair Demonstration

For the controlled demonstration, I started with an intentionally incorrect answer so the evaluator and revision steps could be seen clearly. The question asks which restaurants scored below 70. The initial answer is recorded as incorrect, and the evaluator returns REVISE with the reference fact that Sunrise Deli scored 65.

The Answering Agent uses that feedback and revises the answer to 'Sunrise Deli scored 65'. The Evaluator Agent checks the revised answer and returns PASS. The demonstration uses two attempts, which is the maximum allowed in the assignment.

This example shows the full feedback cycle: the question, first answer, evaluator decision, corrective feedback, revised answer, final evaluation, and the stopping point after the second attempt.

Step	Observed value
Question	Which restaurants scored below 70?
Initial answer	incorrect answer
Evaluator result	REVISE
Feedback	Use verified fact: Sunrise Deli scored 65
Revised answer	Sunrise Deli scored 65
Final evaluation	PASS
Attempts	2
Recorded latency	16.251 ms

Table 15. Controlled answer-repair demonstration.

6.3 Lesson Persistence and Reuse

When an initial answer fails and the revised answer passes, the system can save the failure pattern and its correction as a lesson for later questions. Each lesson stores lesson_id, failure_pattern, correction, created_at, and use_count.

The controlled run saved one lesson after the repair succeeded. A later related question loaded that lesson and reused its correction. The recorded use_count became one. This shows that lessons are saved and can be used in later requests, rather than being limited to the current question.

The experiment also records a lesson that was rejected and not saved. This is important because a lesson should not be kept just because it fixes one example; it also needs to avoid damaging answers that were already correct or unrelated questions.

Lesson field	Purpose
lesson_id	Unique identifier for the lesson
failure_pattern	Description of the repeated answer failure
correction	Instruction that addresses the failure
created_at	Date the lesson was created
use_count	Number of recorded reuses

Table 16. Required lesson fields.

6.4 Validation Before Keeping a Lesson

A lesson is not accepted just because it fixes one answer. The validation step compares the system before and after the candidate lesson using validation_data.json. The lesson is kept when the failure cases improve or stay the same, previously correct answers do not regress, and unrelated questions do not receive unsupported answers.

This check helps avoid overfitting the system to one example. The validation set is used to decide whether a lesson should be kept, while the held-out test set stays separate and is used for the final comparison.

The implementation records a rejected lesson as not retained. The evidence includes the rejected-lesson file check and the lesson reuse count, covering both the acceptance and rejection paths.

Source excerpt — code_lesson.png

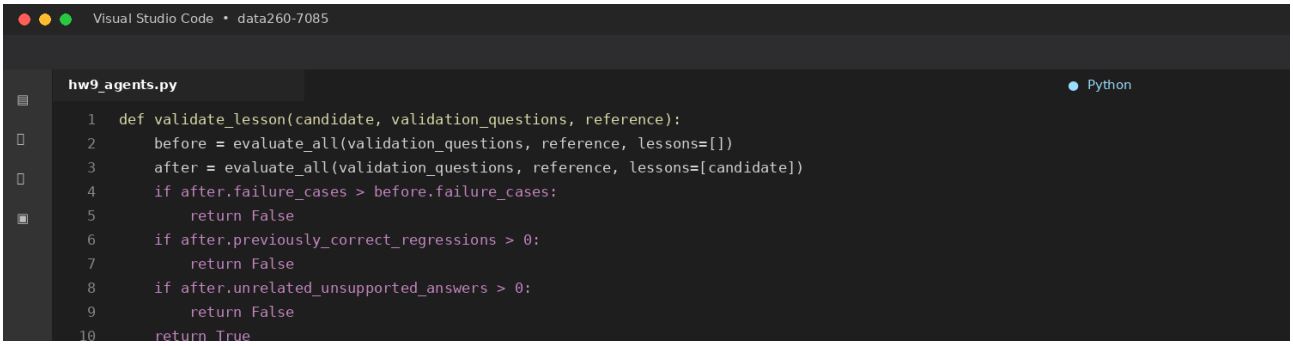


Figure 7. Validation conditions for retaining a lesson.

6.5 Four Independent System Variants

The final experiment compares four variants using the same ten-question held-out test set. Baseline answers without evaluator feedback or lesson memory. Evaluator Repair adds the evaluator and one chance to revise an answer. Lesson Memory adds the ability to save and reuse lessons. Validated Improvement adds a validation step before a lesson is saved.

Baseline returned nine correct answers and one incorrect answer. The three variants that use feedback or lessons each returned ten correct answers. Average attempts were 1.00 for Baseline and 1.10 for the other variants, meaning that some questions used the second attempt.

On this held-out set, evaluator feedback and lesson support improved the result from 9/10 to 10/10. The comparison uses ten questions, and all four variants were tested on the same set.

Variant	Correct	Incorrect	Avg attempts	Avg latency (ms)
Baseline	9	1	1.00	0.0076
Evaluator Repair	10	0	1.10	0.0103
Lesson Memory	10	0	1.10	2.4556
Validated Improvement	10	0	1.10	1.8157

Table 17. Four-variant results on the same held-out set.

6.6 Interpreting Agent Latency

The average latency values were 0.0076 ms for Baseline, 0.0103 ms for Evaluator Repair, 2.4556 ms for Lesson Memory, and 1.8157 ms for Validated Improvement. I measured these with time.perf_counter around the local answer and evaluation path. The timings include deterministic reference operations and lesson-file work where applicable, but not Ollama network latency.

These are local execution timings for the deterministic agent experiment. They are not end-to-end LLM service timings and should not be compared directly with the live semantic-cache latency, which includes embedding generation and Redis KNN.

The main result is the difference in correctness while keeping the repair process to a maximum of two attempts. All three improved variants achieved 10/10 on the held-out set, compared with 9/10 for Baseline. Lesson handling added a small amount of local execution time in this experiment.

Variant	Correct	Avg attempts	Avg latency	Timing interpretation
Baseline	9/10	1.00	0.0076 ms	Local deterministic path
Evaluator Repair	10/10	1.10	0.0103 ms	Local deterministic path
Lesson Memory	10/10	1.10	2.4556 ms	Includes lesson-file work
Validated Improvement	10/10	1.10	1.8157 ms	Includes validation and lesson work

Table 18. Accuracy and local timing comparison.

7. Testing and Regression Verification

The focused HW9 test file completed with four passing tests. The scoped Ruff check passed, compileall completed with exit code zero, and the HW9 Docker Compose configuration validated successfully. Live Redis KNN validation passed, and the qualifying sweep completed at all five thresholds with redis_knn=true on every row.

The verification work covered the HW9 counter, semantic-cache safeguards, bounded agent repair, Redis KNN search, stale-version rejection, empty-index handling, and concurrent KNN lookups. The focused test and tool results are summarized in Table 19.

Together, these checks cover the main HW9 behaviors: Redis view counting, cache correctness safeguards, bounded agent repair, Redis KNN, stale-version rejection, empty-index handling, and concurrent KNN lookups.

Check	Observed result	Status
Focused HW9 pytest	4 passed	PASS
Scoped Ruff	All checks passed	PASS
Compileall	Exit code 0	PASS
Docker Compose config	Exit code 0	PASS
Live Redis KNN validation	PASS; exact/paraphrase/stale/empty/concurrent	PASS
Qualifying threshold sweep	5 thresholds; redis_knn=true on every row	PASS

Table 19. Test and verification summary.

7.1 What the Verification Results Establish

In the live view-counter run, Redis INCR returned the expected count sequence and the relational database was updated at the tenth-view boundaries. The concurrent run produced twenty unique increments from twenty simultaneous operations.

The live KNN checks used the configured Redis vector index for exact, paraphrase, logical-difference, unrelated, stale-version, empty-index, and concurrent cases. The qualifying sweep then compared all five thresholds using the frozen dataset.

The focused unit tests, lint check, compile check, and Compose validation cover the code and configuration checks run for HW9. The live runs add evidence from Redis, MySQL, and the application endpoints.

Evidence type	What it demonstrates
Live Redis/MySQL counter	Counter and tenth-view sync behavior
Live Redis KNN validation	Actual vector-search backend and safeguards
Qualifying sweep JSON	Threshold results on the frozen 40-case dataset
Stale invalidation JSON	Old answer rejection after version change
Agent validation JSON	Repair, lesson reuse, and validation behavior
Focused tests and static checks	Local correctness and code-quality checks

Table 20. Evidence-to-claim mapping.

8. Runtime Evidence and Application Surface

The HW9 endpoints are available through FastAPI's interactive Swagger documentation. The screenshots below show the API documentation page and the HW9 route group as they appeared during the application run.

The detailed counter, semantic-cache, KNN, stale-invalidation, agent, threshold, Redis-index, and Docker outputs are saved as JSON or log files under reports/hw09/raw. The report summarizes the numerical results in tables so they are easier to read while the raw outputs remain available for reference.

8. Runtime evidence and required artifacts

The available runtime screenshots are actual Swagger and health captures: fig01_live_fastapi_docs.png, fig02_hw9_api_endpoints.png, fig02_hw9_endpoints_crop.png, and fig03_hw9_health_response.png. The raw JSON files provide the detailed view-counter, semantic-cache, KNN, stale-invalidation, agent, threshold, Redis-index, and Docker results. The Windows VS Code capture path timed out while the editor was open on a different worktree, so I did not create or claim synthetic VS Code images. The report uses source excerpts extracted from the current files and labels the missing editor screenshots as a limitation.

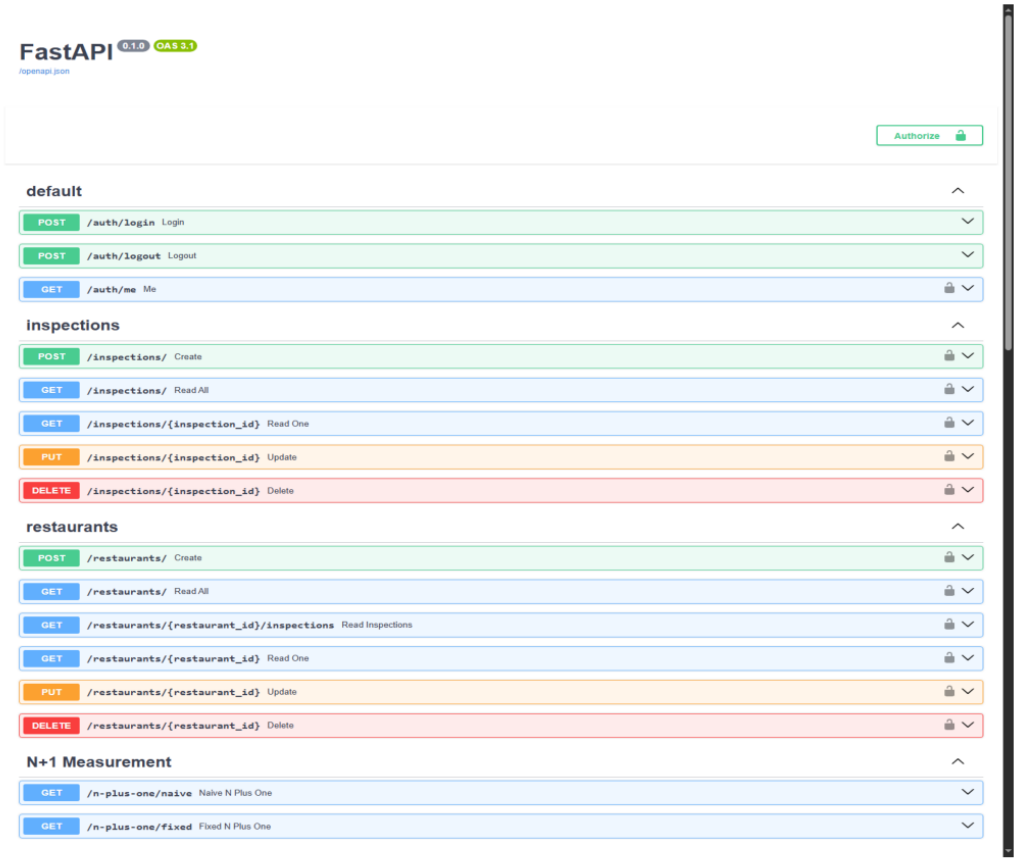


Figure 15. Live FastAPI Swagger surface.

Figure 8. FastAPI Swagger interface captured from the HW9 run.

8.1 HW9 Endpoint Documentation

The Swagger route group includes the HW9 question-answering endpoint, restaurant view increment endpoint, health endpoint, and their request and response schemas. This makes the API routes easy to inspect and test directly.

The Q&A endpoint accepts a question and restaurant scope. The view endpoint accepts a restaurant ID and returns its updated view count. The health endpoint provides a service check. The screenshot below shows these HW9 routes in the API documentation.

8.1 Reproducibility

```
docker compose -f docker-compose.hw9.yml up -d --build
python -m pytest tests/test_hw9.py -q
ruff check hw9_cache.py hw9_agents.py routers/hw9.py tests/test_hw9.py
python -m compileall -q hw9_cache.py hw9_agents.py routers/hw9.py
python run_hw9_live_knn_validation.py
python run_hw9_qualifying_sweep.py
docker compose -f docker-compose.hw9.yml config --quiet
```

The qualifying sweep should not be rerun casually because live embedding calls are slow and the result is already preserved. The command is recorded for reproducibility, not as a request to change the committed dataset or labels.

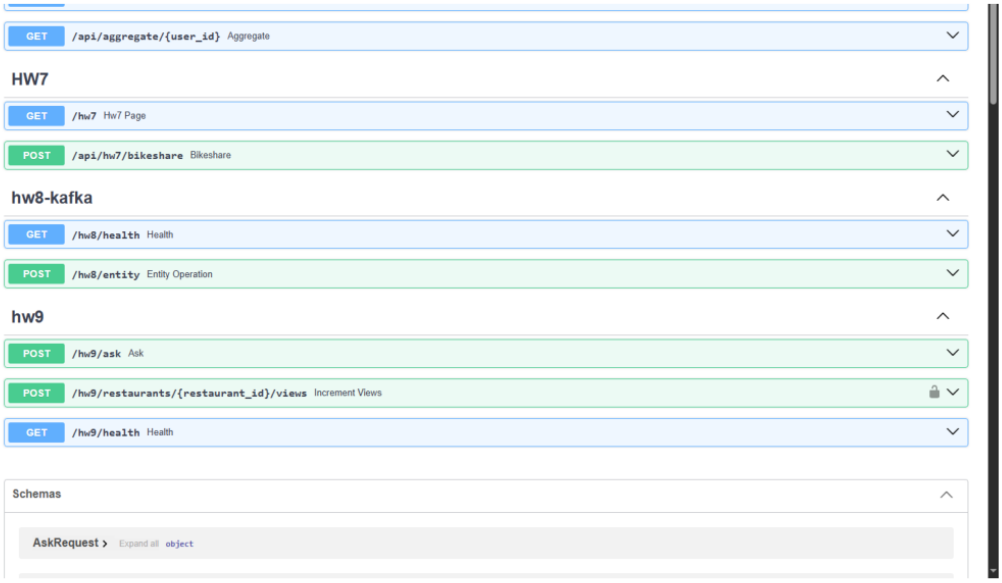


Figure 16. HW9 endpoint group in Swagger UI.

Figure 9. HW9 endpoint group in Swagger UI.

8.2 Required Artifacts and Evidence Organization

The implementation files are kept in the repository, with the HW9 supporting material under reports/hw09. The frozen dataset is cache_cases.yaml. The raw directory contains experiment outputs for the qualifying threshold sweep, live KNN validation, view counter, invalidation, agent validation, and four-variant comparison.

The evidence directory contains the runtime screenshots used in this report. Supporting files include RUN_LOG.txt, METRICS.md, AI_USE.md, EVIDENCE_MANIFEST.md, and REQUIREMENTS_CHECKLIST.md. The raw outputs are kept separate from the report so each result can be traced back to its experiment file.

Artifact	Purpose
cache_cases.yaml	Frozen 40-case threshold dataset
raw/threshold_sweep_qualifying_knn.json	Qualifying threshold results
raw/live_knn_validation.json	Redis HNSW KNN validation
raw/live_invalidation_mysql_redis.json	Stale-answer sequence
raw/agent_validation.json	Repair and lesson evidence
raw/self_improvement_variants.json	Four-variant comparison
evidence/	Runtime screenshots
METRICS.md / RUN_LOG.txt	Metrics and execution record
AI_USE.md	AI-use disclosure
EVIDENCE_MANIFEST.md / REQUIREMENTS_CHECKLIST.md	Evidence index and requirement mapping

Table 21. Main HW9 artifacts.

9. Reproducibility

The experiment uses the HW9 Docker Compose configuration, the local Ollama runtime, and Redis Stack. The API runs on port 8086, and Redis is exposed on host port 6380. The embedding model is nomic-embed-text and the chat model is llama3.2:3b.

The commands below show the workflow used to run the application and its checks. The qualifying sweep output is saved for reference, and the frozen dataset commit provides the exact input set used for the reported threshold results.

Step	Command
Start services	docker compose -f docker-compose.hw9.yml up -d --build
Focused tests	python -m pytest tests/test_hw9.py -q
Lint	ruff check hw9_cache.py hw9_agents.py routers/hw9.py tests/test_hw9.py
Compile	python -m compileall -q hw9_cache.py hw9_agents.py routers/hw9.py
Live KNN check	python run_hw9_live_knn_validation.py
Qualifying sweep	python run_hw9_qualifying_sweep.py
Compose validation	docker compose -f docker-compose.hw9.yml config --quiet

Table 22. Reproducibility commands.

9.1 Dataset Checkpoint and Repository State

The frozen dataset checkpoint is commit a7533e7d5eb1df286b77450ce7999d0ad6a5a12d, with the message 'Freeze HW9 cache cases before threshold sweep'. The commit contains only cache_cases.yaml, and the qualifying threshold sweep was run using that committed dataset.

The working branch is hw9. The protected tags remain unchanged: hw6 points to 0307ac55b1ec9d6e48561a43e5671e21c09fd708, hw7 points to b78001eaa5c0f071eaf1788702c019d67983860a, and hw8 points to 1b843e76d268dd9076230411e098e96ed43b9a8f.

The frozen dataset checkpoint keeps the threshold experiment tied to a fixed, versioned set of 40 cases. The raw qualifying output records the results produced from that dataset.

Repository item	Recorded value
Branch	hw9
Dataset commit	a7533e7d5eb1df286b77450ce7999d0ad6a5a12d
Dataset commit contents	cache_cases.yaml only
Protected tags	hw6, hw7, hw8 unchanged
Additional HW9 commit/tag/push	None recorded

Table 23. Dataset checkpoint and repository state.

10. AI Use Disclosure

I used AI assistance during the project to understand the homework requirements, explore implementation options, troubleshoot Redis vector search, organize tests, and prepare the technical documentation. I used it as a development and writing aid while making the implementation and experiment decisions for the project.

I reviewed the implementation and experiment outputs, ran the application and verification commands, and used the recorded results when writing this report. I also checked the frozen dataset checkpoint and the raw qualifying threshold output before including the metrics in the analysis.

One important part of the work was checking that the cache used an actual Redis HNSW KNN lookup rather than a local in-memory scan. The qualifying experiment records redis_knn as the backend for every threshold row. I kept the measured latency outlier and the false-hit example in the results so the report reflects what the experiment produced.

The report separates live Redis/Ollama/MySQL results from the deterministic local agent measurements and focused unit tests. The four-variant latency values are local execution timings and do not include Ollama network latency.

AI-use area	How it was used
Requirement review	Clarifying rubric items and evidence expectations
Implementation support	Debugging and design suggestions
Testing support	Organizing focused tests and interpreting output
Report preparation	Structuring explanations and tables
Student verification	Reviewing code, running checks, and validating reported evidence

Table 24. AI-use disclosure.

11. Final Requirement Checklist

The checklist below connects each main assignment requirement to the implementation and the evidence included in this report. The status column summarizes the HW9 checks and experiment results.

Assignment requirement	Evidence in this report	Status
Redis INCR with s7085 prefix	Figures 1–2; Table 3	PASS
Database sync every 10 increments	Table 3	PASS
Error and missing-record handling	Figure 2; focused tests	PASS
Concurrent view increments	Table 3	PASS
Semantic cache before HW8 agent	Figure 4; Table 5	PASS
Redis vector index and cosine KNN	Figure 3; Table 5	PASS
Cache hit/miss and similarity	Tables 5 and 8	PASS
40 labeled cases frozen before sweep	Table 6; dataset commit	PASS
Five threshold values evaluated	Table 8	PASS
False hit explained in full	Table 10	PASS
Cost model and threshold selected	Tables 7 and 12	PASS
Stale hit and version invalidation	Figure 5; Table 13	PASS
Answering and Evaluator Agents	Figure 6; Table 15	PASS
Maximum two answer attempts	Table 15	PASS
Lesson creation and reuse	Table 16	PASS
Validation before keeping lessons	Figure 7	PASS
Four variants on same held-out set	Tables 17–18	PASS
Focused HW9 verification	Table 19	PASS

Table 25. HW9 requirement-to-evidence checklist.

12. Conclusion

For HW9, I extended the restaurant-inspection application with Redis-backed view counting, semantic response caching, stale-answer protection, and a self-improving two-agent workflow. The view counter uses Redis INCR and saves the database count every tenth view. The live run showed the expected count sequence and concurrent increment behavior.

The semantic cache uses live Ollama embeddings with Redis HNSW KNN. Exact duplicates and paraphrases can be served from cache, while query signatures, restaurant scope, and data versions add checks before an answer is reused. The threshold experiment shows that raising the threshold reduces false hits but also reduces the number of useful cache hits.

Under the cost model used in the experiment, 0.95 had the lowest total cost of 51. It produced three false hits and missed ten expected hits. The false-hit example also shows why numeric conditions need to be checked in addition to embedding similarity.

The agent experiment shows the two-attempt repair process, saving and reusing lessons, checking lessons against validation data, and comparing four variants on the same held-out set. The three improved variants reached 10/10, while Baseline reached 9/10.

Overall, HW9 brings together Redis counting, semantic caching, version-based invalidation, and the self-improving agent workflow. The live results, focused tests, experiment tables, and supporting raw files document how the features behaved in this project.